

AWS Cloud Processing – Case Studies



www.RKSTech.in

AWS Cloud Processing – Case Studies

Smart Product & Services Solution from AWS



www.RKSTech.in

A leading manufacturer of consumer electronics, InnovateTech, sought to enhance its product offerings by integrating smart technology into its devices. The goal was to create a more connected user experience, improve operational efficiency, and leverage data analytics for product improvement.

Challenges:

- **Data Management:** InnovateTech struggled to handle the vast amount of data generated by smart devices.
- **Scalability:** As the number of connected devices grew, the existing infrastructure could not support the increased load.
- **Security:** Ensuring data privacy and security was critical, especially with user data being transmitted and stored.
- **Time to Market:** The competitive landscape demanded rapid development and deployment of new features.

Solution:

- InnovateTech partnered with AWS to leverage its suite of smart product and service solutions.

Smart Product & Services Solution from AWS



www.RKSTech.in

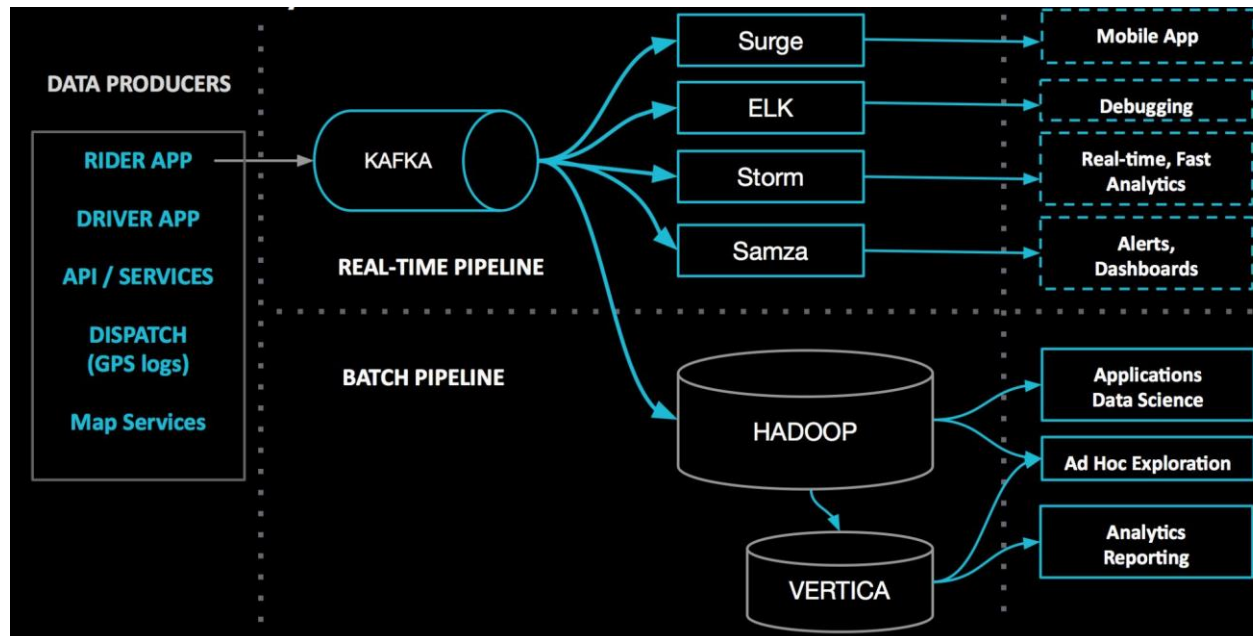
The implementation included the following key components:

- **AWS IoT Core** to enabled secure communication between devices and the cloud.
 - **Amazon S3 and DynamoDB** - Provided a reliable and scalable storage solution for vast amounts of device-generated data.
 - **Fast Access:** DynamoDB was utilized for low-latency access to data, crucial for real-time applications.
 - **AWS Lambda:** InnovateTech implemented serverless computing to run code in response to events without provisioning servers, allowing for efficient scaling and cost management.
 - **AWS Kinesis** Stream Processing for Real-Time Data Processing, facilitating instant insights and actions
 - **Amazon Sage Maker for Machine Learning Integration:** Used to build, train, and deploy machine learning models for predictive maintenance and user behavior analysis, enabling the development of personalized user experiences.
 - **AWS Security Solutions for Data Protection:** Implemented AWS Identity and Access Management (IAM) for user authentication and AWS Key Management Service (KMS) for data encryption, ensuring the security of sensitive user information.
-
- <https://aws.amazon.com/iot-core/>
 - <https://docs.aws.amazon.com/iot/latest/developerguide/what-is-aws-iot.html>
 - <https://aws.amazon.com/industrial/smart-products-and-services/>
 - <https://aws.amazon.com/solutions/guidance/connected-products-with-simpleiot-on-aws/>

uReplicator: Uber Engineering's Robust Apache Kafka Replicator



www.RKSTech.in



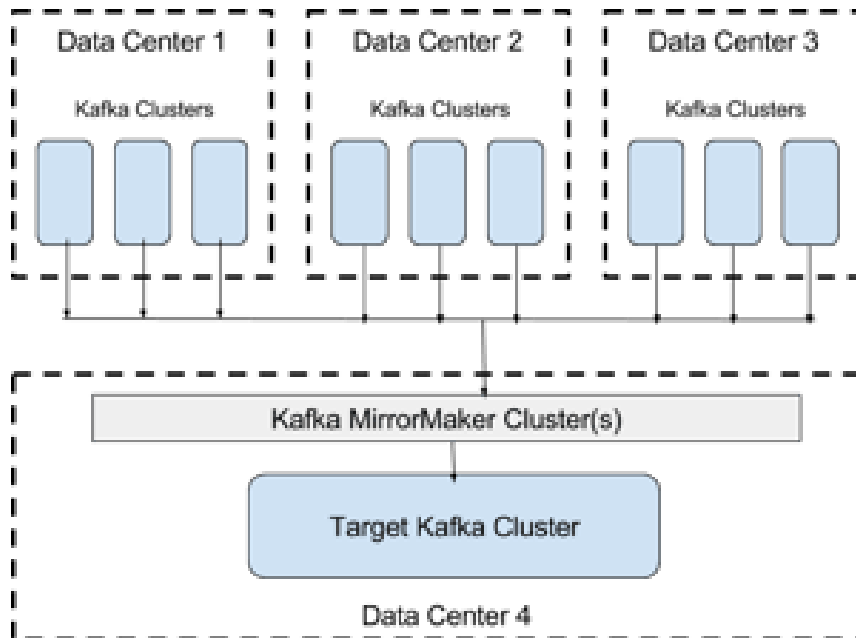
- Uses **Apache Kafka** as a message bus for connecting different parts of the ecosystem .
 - Collects system, app & event logs from the rider & driver apps.
 - Makes this data available to multiple downstream consumers
 - Data pipeline includes both real time and batch data.
 - Real time data is like computing metrics, debugging, alerting, and dashboarding.
 - Batch pipeline data is more exploratory, such as ETL into Apache Hadoop and HP Vertica.

• <https://www.uber.com/en-IN/blog/ureplicator-apache-kafka-replicator/>

uReplicator: Uber Engineering's Robust Apache Kafka Replicator



www.RKSTech.in



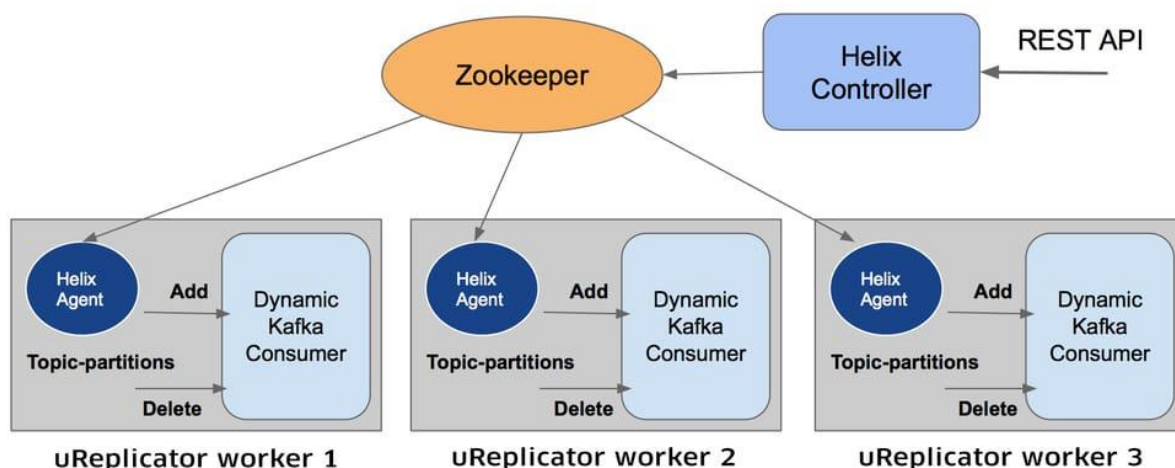
- Given the large-scale use of Kafka within Uber, Uber ended up having multiple clusters in different data centers,
 - There is often need of a global view of the data.
 - For this they need to gather information from all data centers and analyze it in one place.
- Historically, Uber has used the open source MirrorMaker tool shipped with the Kafka package for this.
- Mirror Maker uses a high-level Kafka consumer to fetch the data from the source cluster, which it feeds into a Kafka producer to dump it into the destination cluster. This leads to many scalability issues:
 - Expensive rebalancing: Violates Uber's end-to-end latency guarantee
 - Difficulty adding topics: requires a static whitelist of topics that can be added, which also leads to expensive rebalancing
 - Possible data loss: Automatic offset commits are unreliable
 - Metadata sync issues
- Ultimately, Uber developed a solution named uReplicator to solve these issues:

• <https://www.uber.com/en-IN/blog/ureplicator-apache-kafka-replicator/>



www.RKSTech.in

uReplicator: Uber Engineering's Robust Apache Kafka Replicator



• <https://www.uber.com/en-IN/blog/ureplicator-apache-kafka-replicator/>

uReplicator's various components work in different ways toward reliability and stability.

- Helix uReplicator Controller
- uReplicator Worker replicates topic partitions from source cluster to destination cluster, and avoids rebalancing
- Helix Agent for each uReplicator notifies the Dynamic Kafka Consumer of changes, to add/remove topic-partitions
- Dynamic Kafka Consumer instance on each uReplicator worker, which. Removes the rebalance part, and Adds a mechanism to add/delete topic-partitions on the fly

The benefits of uReplicator include:

- Stability: Rebalancing now happens only during startup and node addition/deletion, and only affects a few partitions
- Easier Scalability: Adding a new node to an existing cluster is now much simpler
- Easier Operation: No need to restart the cluster when adding/deleting/expanding Kafka topics
- Zero Data Loss: uReplicator commits checkpoints only after the data has been persisted on the destination cluster
- Given the large-scale use of Kafka within Uber,